

Week 6

- Convert column values into **numeric (number) format**.
- Columns that should be numeric may contain:
- Text values
- Symbols
- Missing values
- Mixed data types

carbo	sugars	potass
23	6	120
15	8	—
twenty	5	90
18g	7	110

errors='coerce'

This is the key concept.

It means:

If any value cannot be converted → replace it with **NaN (Not a Number)**.

Handling Missing Values

- `df[column] = df[column].fillna(df[column].mean())`
- If any value in that column is **missing (NaN)**, replace it with the **column mean**.
- **Why mean imputation? Why rounding?**
- Cleaner dataset
- Easier visualization
- Standardized precision

.round(2)

Used because:

- Keeps distribution balanced
- Avoids deleting rows
- Common in ML preprocessing

Noisy data / outlier detection and treatment function using the **IQR** method

- Detects **outliers (noisy data)** in numerical columns and replaces them with the **median**.
- Outliers can distort:
 - Mean
 - Correlation
 - ML model performance
 - Statistical analysis

- IQR = Interquartile Range

IQR tells us:

“How spread out the middle 50% of the data is.”

It ignores very high and very low values (outliers).

So it measures **true data spread without noise**.

Quartile	Symbol	Meaning
1st Quartile	Q1	25% data below
2nd Quartile	Q2	Median (50%)
3rd Quartile	Q3	75% data below

When data is sorted, it is divided into 3 equal parts:

IQR Formula

$$\text{IQR} = Q3 - Q1$$

It measures the range between 25% and 75% data.

- Step 1:

$Q1 = df[column].quantile(0.25)$

$Q3 = df[column].quantile(0.75)$

Step 2:

$IQR = Q3 - Q1$ Measures spread of middle 50% data.

Threshold

Interpretation

1.5

Mild outliers
(standard)

Step 3:

$lower_bound = Q1 - threshold * IQR$

3.0

Extreme outliers

$upper_bound = Q3 + threshold * IQR$

<1.5

Very strict detection

- These bounds define the **normal data range**.
- Any value:
- Below $lower_bound \rightarrow$ Lower outlier
- Above $upper_bound \rightarrow$ Upper outlier
- So this formula is used to **separate normal data from noisy / anomalous data**.

Why threshold?

- Because not all variation is noise.
- We allow some buffer beyond $Q1$ and $Q3$.
- Threshold controls sensitivity.

Replace Outliers

```
df[column] = df[column].apply(  
    lambda x: df[column].median()  
    if x < lower_bound or x > upper_bound else x  
)
```

- For each value x:
- If outside bounds → replace with median
- Else → keep original
- Median is robust to outliers.
- Example:

[2, 3, 4, 5, 6, 7, 50]

Median = 5

Removes duplicates and returns distinct cereal names.

- ['Cornflakes', 'Oats', 'Cornflakes', 'Muesli']
- After .unique():
- ['Cornflakes', 'Oats', 'Muesli']
- **Count Unique Cereals**
- len(unique_brands)
- **print("The number of cereals in this file is",**
- **len(unique_brands),**
- **"and they are:\n",**
- **unique_brands)**
- The number of cereals in this file is 3 and they are:
- ['Cornflakes' 'Oats' 'Muesli']

- You are identifying:
- Best cereal (highest rating)
- Worst cereal (lowest rating)
- `best = df.loc[df["rating"].idxmax(), "name"]`
- `worst = df.loc[df["rating"].idxmin(), "name"]`
- `print("The best cereal is", best, "and the worst cereal is", worst, ".")`

`df.loc[index, "name"]`

Let ratings be:

- `.loc` accesses data by:
- Row index
- Column label

$$R = \{r_1, r_2, r_3, \dots, r_n\}$$

Then:

1. Find row index of highest rating

$$Best = \arg \max(R)$$

2. Go to that row

$$Worst = \arg \min(R)$$

3. Extract cereal name

Where `argmax` returns index of maximum value.

How many cereals are placed on each supermarket display shelf and visualizing it using a bar chart.

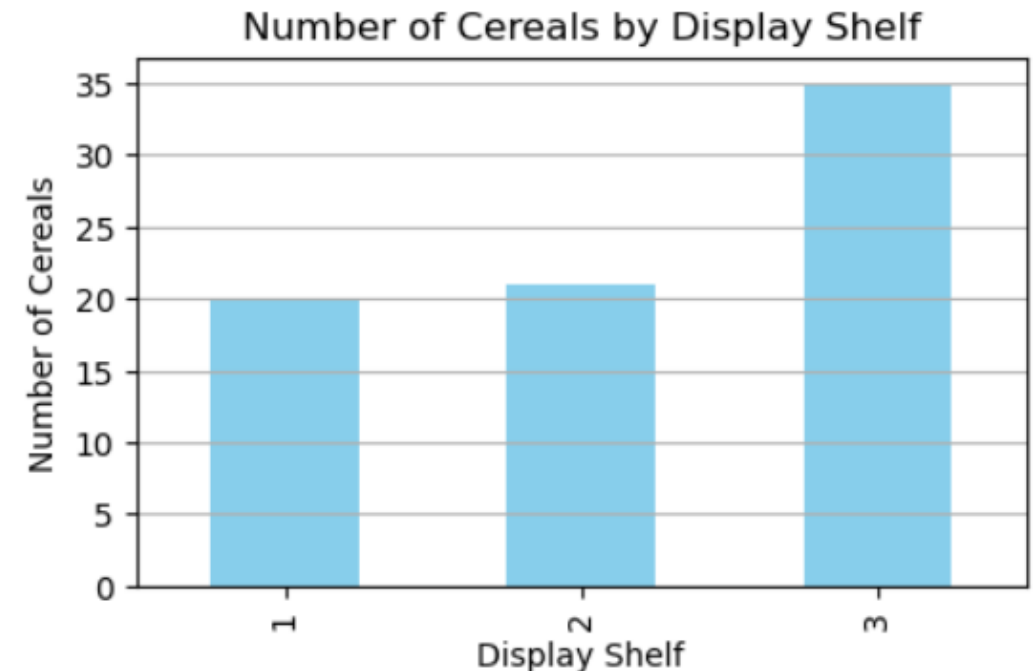
- `shelf_counts = df['shelf'].value_counts()`

Counts how many cereals are on each shelf.

```
shelf_counts_df.plot(kind='bar',  
                      x='Shelf',  
                      y='Number of Cereals',  
                      color='skyblue',  
                      legend=False,  
                      figsize=(5, 3))
```

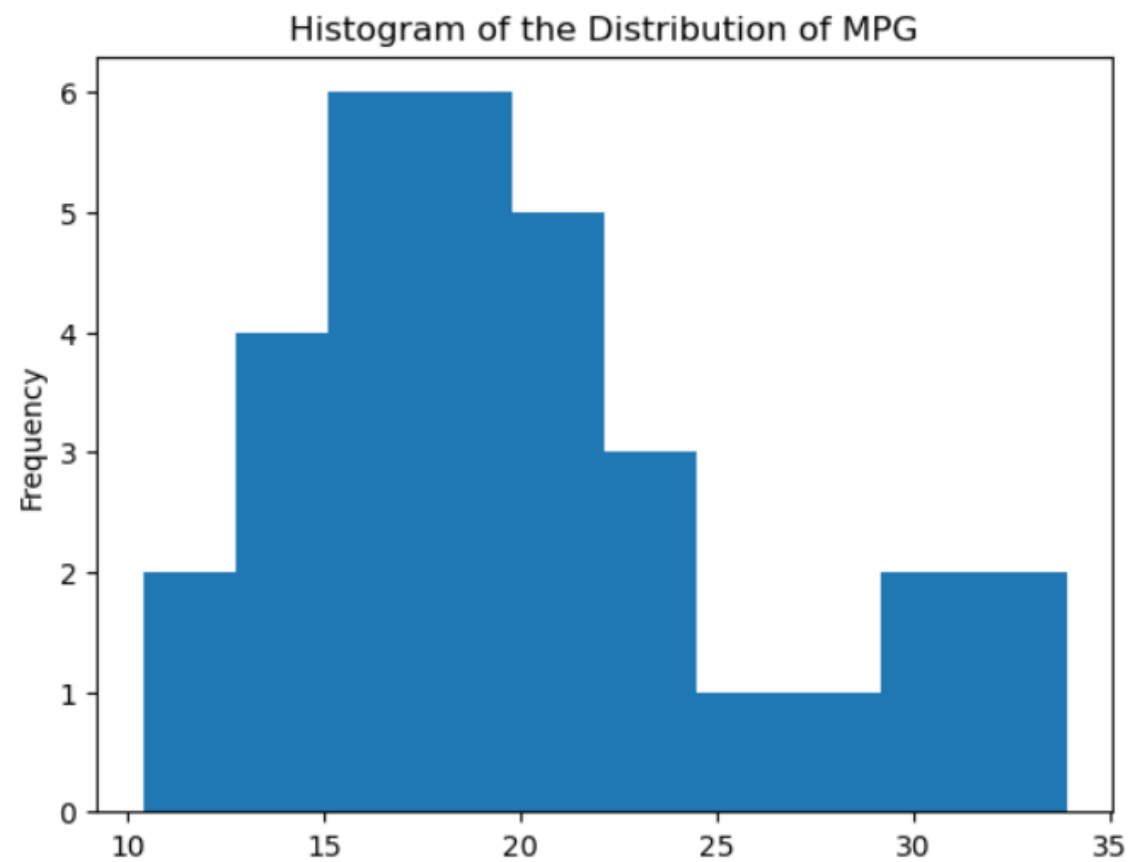
```
plt.title('Number of Cereals by Display Shelf')  
plt.xlabel('Display Shelf')  
plt.ylabel('Number of Cereals')
```

	Shelf	Number of Cereals
0	1	20
1	2	21
2	3	35



```
df["mpg"].plot(kind='hist', bins=10, title="Histogram of the Distribution of MPG")
```

```
<Axes: title={'center': 'Histogram of the Distribution of MPG'}, ylabel='Frequency'>
```



- A **box plot** is a statistical graph that summarizes the distribution of a numerical variable using:
- Quartiles
- Median
- Spread
- Outliers
- It is built directly from the **IQR concept**

- `df["mpg"].plot(kind='box')`

This creates a boxplot for the **mpg column**.
(mpg = miles per gallon — fuel efficiency).

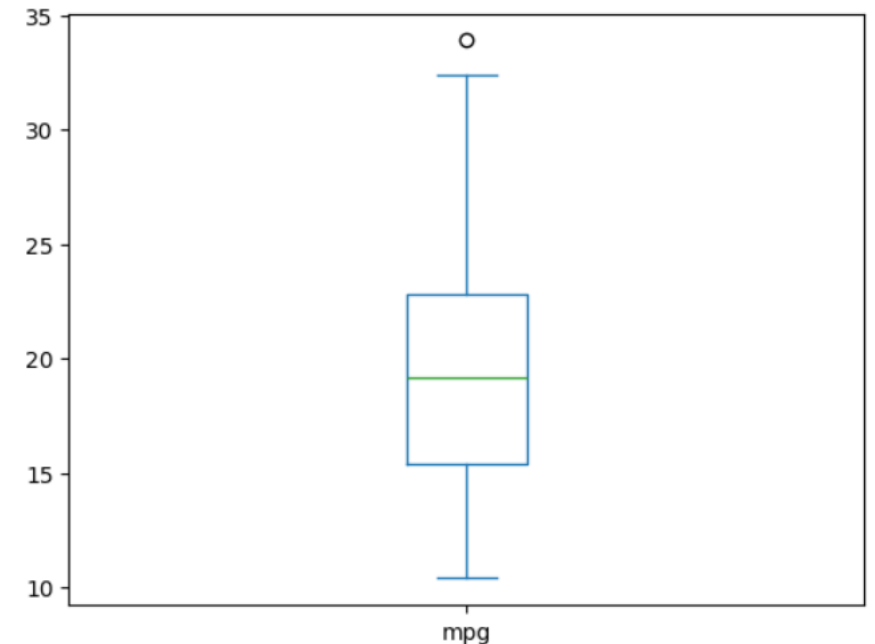
The line inside the box is the **median (Q2)**.

≈ **19 mpg**

Meaning half vehicles have mpg below 19 and half above.

```
df["mpg"].plot(kind='box')
```

<Axes: >



Represents the **minimum non-outlier value**.

Represents the **maximum non-outlier value**.